LOW-LEVEL DESIGN (LLD) DOCUMENT

AI-Based Monitoring Solution for Oil & Gas Industry

Document Version: 1.0 **Date**: December 2025 **Project Code**: OG-AI-MON-2025 **Classification**: Confidential

---

1. DOCUMENT OVERVIEW

1.1 Purpose

This Low-Level Design (LLD) document provides detailed technical specifications for implementing the AI-Based Monitoring Solution. It includes database schemas, API specifications, algorithm details, configuration parameters, and code structures.

1.2 Scope

In Scope	Out of Scope
Database schemas and indexes	Business requirements
API endpoint specifications	High-level architecture
Data models and classes	Project management
Configuration parameters	Training materials
Algorithm pseudocode	User documentation
Message formats	

---

2. DATABASE DESIGN

2.1 PostgreSQL Schema

2.1.1 Core Tables

```
-- ============================================================
-- ASSET MANAGEMENT SCHEMA
-- ============================================================

CREATE SCHEMA IF NOT EXISTS asset_mgmt;

-- Assets Table
CREATE TABLE asset_mgmt.assets (
    asset_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    asset_tag VARCHAR(50) UNIQUE NOT NULL,
    name VARCHAR(255) NOT NULL,
    asset_type VARCHAR(50) NOT NULL,  -- pump, compressor, motor, valve, tank
    manufacturer VARCHAR(100),
    model VARCHAR(100),
    serial_number VARCHAR(100),
    installation_date DATE,
    location_id UUID REFERENCES asset_mgmt.locations(location_id),
    parent_asset_id UUID REFERENCES asset_mgmt.assets(asset_id),
```

```
    criticality VARCHAR(20) DEFAULT 'medium',  -- low, medium, high, critical
    status VARCHAR(20) DEFAULT 'operational',  -- operational, degraded, failed, maintenance
    specifications JSONB,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW(),
    created_by UUID,
    updated_by UUID
);

-- Indexes
CREATE INDEX idx_assets_location ON asset_mgmt.assets(location_id);
CREATE INDEX idx_assets_type ON asset_mgmt.assets(asset_type);
CREATE INDEX idx_assets_status ON asset_mgmt.assets(status);
CREATE INDEX idx_assets_criticality ON asset_mgmt.assets(criticality);

-- Locations Table
CREATE TABLE asset_mgmt.locations (
    location_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    location_code VARCHAR(50) UNIQUE NOT NULL,
    name VARCHAR(255) NOT NULL,
    facility_type VARCHAR(50),  -- refinery, offshore, pipeline, gas_plant
    parent_location_id UUID REFERENCES asset_mgmt.locations(location_id),
    latitude DECIMAL(10, 8),
    longitude DECIMAL(11, 8),
    timezone VARCHAR(50) DEFAULT 'UTC',
    hazardous_area_class VARCHAR(20),  -- zone0, zone1, zone2, safe
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Sensors Table
CREATE TABLE asset_mgmt.sensors (
    sensor_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    sensor_tag VARCHAR(50) UNIQUE NOT NULL,
    asset_id UUID REFERENCES asset_mgmt.assets(asset_id),
    sensor_type VARCHAR(50) NOT NULL,  -- vibration, temperature, pressure, flow, level
    manufacturer VARCHAR(100),
    model VARCHAR(100),
    unit_of_measure VARCHAR(20),
    min_value DECIMAL(15, 5),
    max_value DECIMAL(15, 5),
    alarm_low DECIMAL(15, 5),
    alarm_high DECIMAL(15, 5),
    sampling_rate_hz DECIMAL(10, 2),
    communication_protocol VARCHAR(50),  -- opc-ua, modbus, mqtt, 4-20ma
    opc_node_id VARCHAR(255),
    status VARCHAR(20) DEFAULT 'active',
    calibration_date DATE,
    next_calibration_date DATE,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_sensors_asset ON asset_mgmt.sensors(asset_id);
CREATE INDEX idx_sensors_type ON asset_mgmt.sensors(sensor_type);
```

```sql
-- ============================================================
-- ALERT MANAGEMENT SCHEMA
-- ============================================================

CREATE SCHEMA IF NOT EXISTS alerts;

-- Alert Definitions
CREATE TABLE alerts.alert_definitions (
    definition_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(255) NOT NULL,
    description TEXT,
    alert_type VARCHAR(50) NOT NULL,  -- anomaly, prediction, threshold, cv_detection
    severity VARCHAR(20) NOT NULL,  -- info, warning, critical, emergency
    source_model VARCHAR(100),  -- model that generates this alert
    enabled BOOLEAN DEFAULT TRUE,
    notification_channels TEXT[],  -- email, sms, push, dashboard
    auto_create_work_order BOOLEAN DEFAULT FALSE,
    escalation_minutes INTEGER DEFAULT 30,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Alert Instances
CREATE TABLE alerts.alert_instances (
    alert_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    definition_id UUID REFERENCES alerts.alert_definitions(definition_id),
    asset_id UUID REFERENCES asset_mgmt.assets(asset_id),
    sensor_id UUID REFERENCES asset_mgmt.sensors(sensor_id),
    severity VARCHAR(20) NOT NULL,
    title VARCHAR(255) NOT NULL,
    description TEXT,
    confidence_score DECIMAL(5, 4),  -- 0.0000 to 1.0000
    model_version VARCHAR(50),
    triggered_at TIMESTAMPTZ NOT NULL,
    acknowledged_at TIMESTAMPTZ,
    acknowledged_by UUID,
    resolved_at TIMESTAMPTZ,
    resolved_by UUID,
    resolution_notes TEXT,
    status VARCHAR(20) DEFAULT 'open',  -- open, acknowledged, resolved, false_positive
    related_data JSONB,  -- sensor values, predictions, etc.
    created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_alerts_asset ON alerts.alert_instances(asset_id);
CREATE INDEX idx_alerts_status ON alerts.alert_instances(status);
CREATE INDEX idx_alerts_triggered ON alerts.alert_instances(triggered_at DESC);
CREATE INDEX idx_alerts_severity ON alerts.alert_instances(severity);

-- ============================================================
-- ML MODEL MANAGEMENT SCHEMA
-- ============================================================

CREATE SCHEMA IF NOT EXISTS ml_models;
```

```sql
-- Model Registry
CREATE TABLE ml_models.model_registry (
    model_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    model_name VARCHAR(100) NOT NULL,
    model_version VARCHAR(50) NOT NULL,
    model_type VARCHAR(50) NOT NULL,  -- classification, regression, anomaly, detection
    framework VARCHAR(50),  -- pytorch, tensorflow, onnx, xgboost
    description TEXT,
    input_schema JSONB,
    output_schema JSONB,
    metrics JSONB,  -- accuracy, precision, recall, etc.
    hyperparameters JSONB,
    artifact_path VARCHAR(500),  -- s3://bucket/path/model.onnx
    deployed_edge BOOLEAN DEFAULT FALSE,
    deployed_cloud BOOLEAN DEFAULT FALSE,
    status VARCHAR(20) DEFAULT 'staging',  -- staging, production, archived
    created_at TIMESTAMPTZ DEFAULT NOW(),
    created_by UUID,
    UNIQUE(model_name, model_version)
);

-- Model Deployments
CREATE TABLE ml_models.model_deployments (
    deployment_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    model_id UUID REFERENCES ml_models.model_registry(model_id),
    deployment_target VARCHAR(50) NOT NULL,  -- edge_node_001, cloud_triton
    deployment_type VARCHAR(20),  -- edge, cloud
    endpoint_url VARCHAR(500),
    replicas INTEGER DEFAULT 1,
    status VARCHAR(20) DEFAULT 'pending',  -- pending, running, failed, stopped
    deployed_at TIMESTAMPTZ,
    health_check_url VARCHAR(500),
    last_health_check TIMESTAMPTZ,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Model Predictions Log
CREATE TABLE ml_models.prediction_log (
    prediction_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    model_id UUID REFERENCES ml_models.model_registry(model_id),
    asset_id UUID REFERENCES asset_mgmt.assets(asset_id),
    input_features JSONB,
    prediction_output JSONB,
    confidence_score DECIMAL(5, 4),
    inference_time_ms INTEGER,
    ground_truth JSONB,  -- filled later for validation
    feedback VARCHAR(20),  -- correct, incorrect, null
    predicted_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_predictions_model ON ml_models.prediction_log(model_id);
CREATE INDEX idx_predictions_asset ON ml_models.prediction_log(asset_id);
CREATE INDEX idx_predictions_time ON ml_models.prediction_log(predicted_at DESC);
```

```sql
-- Partition by month
CREATE TABLE ml_models.prediction_log_2025_01 PARTITION OF ml_models.prediction_log
    FOR VALUES FROM ('2025-01-01') TO ('2025-02-01');

-- ============================================================
-- RAG KNOWLEDGE BASE SCHEMA
-- ============================================================

CREATE SCHEMA IF NOT EXISTS knowledge;

-- Document Metadata
CREATE TABLE knowledge.documents (
    document_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    file_name VARCHAR(255) NOT NULL,
    file_path VARCHAR(500),
    file_type VARCHAR(20),  -- pdf, docx, cad, html
    document_type VARCHAR(50),  -- manual, sop, pid, incident_report, standard
    title VARCHAR(500),
    description TEXT,
    equipment_tags TEXT[],  -- related equipment
    version VARCHAR(20),
    effective_date DATE,
    expiry_date DATE,
    author VARCHAR(255),
    department VARCHAR(100),
    language VARCHAR(10) DEFAULT 'en',
    page_count INTEGER,
    word_count INTEGER,
    file_size_bytes BIGINT,
    checksum_sha256 VARCHAR(64),
    indexed_at TIMESTAMPTZ,
    status VARCHAR(20) DEFAULT 'pending',  -- pending, indexed, failed
    created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_documents_type ON knowledge.documents(document_type);
CREATE INDEX idx_documents_equipment ON knowledge.documents USING GIN(equipment_tags);

-- Document Chunks
CREATE TABLE knowledge.chunks (
    chunk_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    document_id UUID REFERENCES knowledge.documents(document_id),
    chunk_index INTEGER NOT NULL,
    content TEXT NOT NULL,
    token_count INTEGER,
    page_number INTEGER,
    section_title VARCHAR(255),
    embedding_model VARCHAR(100),
    embedding_id VARCHAR(100),  -- ID in Milvus
    metadata JSONB,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_chunks_document ON knowledge.chunks(document_id);
```

```sql
-- RAG Query Log
CREATE TABLE knowledge.query_log (
    query_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID,
    query_text TEXT NOT NULL,
    query_embedding_id VARCHAR(100),
    retrieved_chunk_ids UUID[],
    retrieved_scores DECIMAL(5,4)[],
    generated_response TEXT,
    response_tokens INTEGER,
    latency_ms INTEGER,
    feedback VARCHAR(20),  -- helpful, not_helpful, null
    feedback_notes TEXT,
    session_id UUID,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_query_log_user ON knowledge.query_log(user_id);
CREATE INDEX idx_query_log_time ON knowledge.query_log(created_at DESC);
```

2.2 InfluxDB Schema

2.2.1 Measurements

```
# Bucket: sensor_data (7 days retention)
# Bucket: sensor_data_downsampled (1 year retention)
# Bucket: ml_features (30 days retention)
# Bucket: predictions (1 year retention)

# Raw Sensor Data
measurement: sensor_readings
  tags:
    - asset_id (string)
    - sensor_id (string)
    - sensor_type (string)
    - location_id (string)
  fields:
    - value (float)
    - quality (int)  # 0=bad, 1=uncertain, 2=good
    - raw_value (float)

# Vibration Waveform (high-frequency)
measurement: vibration_waveform
  tags:
    - asset_id (string)
    - sensor_id (string)
    - axis (string)  # x, y, z
  fields:
    - samples (array of float, 8192 points)
    - sampling_rate (float)
    - trigger_level (float)

# Extracted Features
measurement: vibration_features
```

```
  tags:
    - asset_id (string)
    - sensor_id (string)
  fields:
    - rms (float)
    - peak (float)
    - peak_to_peak (float)
    - crest_factor (float)
    - kurtosis (float)
    - skewness (float)
    - fft_1x (float)  # 1x rotational frequency
    - fft_2x (float)
    - fft_3x (float)
    - bpfo (float)  # bearing outer race
    - bpfi (float)  # bearing inner race
    - bsf (float)   # ball spin frequency
    - ftf (float)   # cage frequency

# ML Predictions
measurement: predictions
  tags:
    - asset_id (string)
    - model_name (string)
    - model_version (string)
  fields:
    - rul_days (float)  # Remaining Useful Life
    - anomaly_score (float)
    - confidence (float)
    - health_index (float)
    - predicted_failure_mode (string)

# Alerts
measurement: alert_events
  tags:
    - asset_id (string)
    - alert_type (string)
    - severity (string)
  fields:
    - description (string)
    - confidence (float)
    - acknowledged (bool)
```

2.2.2 Continuous Queries

```
// Downsample sensor data from 1s to 1m
option task = {name: "downsample_1m", every: 1m}

from(bucket: "sensor_data")
  |> range(start: -1m)
  |> filter(fn: (r) => r._measurement == "sensor_readings")
  |> aggregateWindow(every: 1m, fn: mean, createEmpty: false)
  |> to(bucket: "sensor_data_downsampled")

// Calculate hourly statistics
```

```
option task = {name: "hourly_stats", every: 1h}

from(bucket: "sensor_data")
  |> range(start: -1h)
  |> filter(fn: (r) => r._measurement == "sensor_readings")
  |> aggregateWindow(
      every: 1h,
      fn: (column, tables=<-) => tables
        |> reduce(
            fn: (r, accumulator) => ({
              count: accumulator.count + 1,
              sum: accumulator.sum + r._value,
              min: if r._value < accumulator.min then r._value else accumulator.min,
              max: if r._value > accumulator.max then r._value else accumulator.max
            }),
            identity: {count: 0, sum: 0.0, min: 999999.0, max: -999999.0}
        )
    )
  |> to(bucket: "sensor_data_hourly")
```

2.3 Milvus Vector Schema

```
# Collection: document_embeddings
{
    "collection_name": "document_embeddings",
    "fields": [
        {
            "name": "chunk_id",
            "dtype": DataType.VARCHAR,
            "max_length": 36,
            "is_primary": True
        },
        {
            "name": "document_id",
            "dtype": DataType.VARCHAR,
            "max_length": 36
        },
        {
            "name": "document_type",
            "dtype": DataType.VARCHAR,
            "max_length": 50
        },
        {
            "name": "embedding",
            "dtype": DataType.FLOAT_VECTOR,
            "dim": 1024  # BGE-large dimension
        },
        {
            "name": "content_preview",
            "dtype": DataType.VARCHAR,
            "max_length": 500
        }
    ],
    "index": {
```

```
        "field_name": "embedding",
        "index_type": "IVF_PQ",
        "metric_type": "COSINE",
        "params": {
            "nlist": 1024,
            "m": 16,
            "nbits": 8
        }
    }
}

# Collection: query_cache
{
    "collection_name": "query_cache",
    "fields": [
        {
            "name": "query_hash",
            "dtype": DataType.VARCHAR,
            "max_length": 64,
            "is_primary": True
        },
        {
            "name": "query_embedding",
            "dtype": DataType.FLOAT_VECTOR,
            "dim": 1024
        },
        {
            "name": "cached_response",
            "dtype": DataType.VARCHAR,
            "max_length": 10000
        },
        {
            "name": "created_at",
            "dtype": DataType.INT64
        }
    ]
}
```

---

3. API SPECIFICATIONS

3.1 Asset Service API

3.1.1 OpenAPI Specification

```yaml
openapi: 3.0.3
info:
  title: Asset Service API
  version: 1.0.0
  description: API for managing industrial assets and sensors

servers:
  - url: https://api.oilgas-ai.internal/v1
    description: Production
```

```yaml
security:
  - bearerAuth: []

paths:
  /assets:
    get:
      summary: List all assets
      parameters:
        - name: location_id
          in: query
          schema:
            type: string
            format: uuid
        - name: asset_type
          in: query
          schema:
            type: string
            enum: [pump, compressor, motor, valve, tank]
        - name: status
          in: query
          schema:
            type: string
            enum: [operational, degraded, failed, maintenance]
        - name: page
          in: query
          schema:
            type: integer
            default: 1
        - name: limit
          in: query
          schema:
            type: integer
            default: 50
            maximum: 100
      responses:
        '200':
          description: List of assets
          content:
            application/json:
              schema:
                type: object
                properties:
                  data:
                    type: array
                    items:
                      $ref: '#/components/schemas/Asset'
                  pagination:
                    $ref: '#/components/schemas/Pagination'

    post:
      summary: Create new asset
      requestBody:
        required: true
```

```yaml
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/AssetCreate'
    responses:
      '201':
        description: Asset created
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Asset'
      '400':
        description: Validation error
      '409':
        description: Asset tag already exists

/assets/{asset_id}:
  get:
    summary: Get asset by ID
    parameters:
      - name: asset_id
        in: path
        required: true
        schema:
          type: string
          format: uuid
    responses:
      '200':
        description: Asset details
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/AssetDetail'
      '404':
        description: Asset not found

/assets/{asset_id}/health:
  get:
    summary: Get asset health status
    parameters:
      - name: asset_id
        in: path
        required: true
        schema:
          type: string
          format: uuid
    responses:
      '200':
        description: Asset health information
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/AssetHealth'
```

```yaml
/assets/{asset_id}/sensors:
  get:
    summary: List sensors for asset
    parameters:
      - name: asset_id
        in: path
        required: true
        schema:
          type: string
          format: uuid
    responses:
      '200':
        description: List of sensors
        content:
          application/json:
            schema:
              type: array
              items:
                $ref: '#/components/schemas/Sensor'

/assets/{asset_id}/predictions:
  get:
    summary: Get predictions for asset
    parameters:
      - name: asset_id
        in: path
        required: true
        schema:
          type: string
          format: uuid
      - name: model_name
        in: query
        schema:
          type: string
      - name: start_time
        in: query
        schema:
          type: string
          format: date-time
      - name: end_time
        in: query
        schema:
          type: string
          format: date-time
    responses:
      '200':
        description: Predictions for asset
        content:
          application/json:
            schema:
              type: array
              items:
                $ref: '#/components/schemas/Prediction'
```

```yaml
components:
  schemas:
    Asset:
      type: object
      properties:
        asset_id:
          type: string
          format: uuid
        asset_tag:
          type: string
        name:
          type: string
        asset_type:
          type: string
        status:
          type: string
        criticality:
          type: string
        location_id:
          type: string
          format: uuid
        health_index:
          type: number
          format: float
          minimum: 0
          maximum: 1

    AssetCreate:
      type: object
      required:
        - asset_tag
        - name
        - asset_type
        - location_id
      properties:
        asset_tag:
          type: string
          maxLength: 50
        name:
          type: string
          maxLength: 255
        asset_type:
          type: string
          enum: [pump, compressor, motor, valve, tank]
        location_id:
          type: string
          format: uuid
        manufacturer:
          type: string
        model:
          type: string
        criticality:
          type: string
          enum: [low, medium, high, critical]
```

```yaml
      default: medium
    specifications:
      type: object

AssetHealth:
  type: object
  properties:
    asset_id:
      type: string
      format: uuid
    health_index:
      type: number
      description: "0-1 scale, 1 = healthy"
    status:
      type: string
    rul_days:
      type: integer
      description: "Remaining Useful Life in days"
    anomaly_score:
      type: number
    last_prediction:
      type: string
      format: date-time
    active_alerts:
      type: integer
    sensors:
      type: array
      items:
        type: object
        properties:
          sensor_id:
            type: string
          status:
            type: string
          current_value:
            type: number
          unit:
            type: string

Sensor:
  type: object
  properties:
    sensor_id:
      type: string
      format: uuid
    sensor_tag:
      type: string
    sensor_type:
      type: string
    unit_of_measure:
      type: string
    current_value:
      type: number
    status:
```

```
          type: string
        last_reading:
          type: string
          format: date-time

    Prediction:
      type: object
      properties:
        prediction_id:
          type: string
          format: uuid
        model_name:
          type: string
        model_version:
          type: string
        rul_days:
          type: integer
        confidence:
          type: number
        health_index:
          type: number
        predicted_at:
          type: string
          format: date-time

    Pagination:
      type: object
      properties:
        page:
          type: integer
        limit:
          type: integer
        total:
          type: integer
        pages:
          type: integer

  securitySchemes:
    bearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT
```

3.2 RAG Service API

```
openapi: 3.0.3
info:
  title: RAG Knowledge Assistant API
  version: 1.0.0

paths:
  /rag/query:
    post:
      summary: Query the knowledge base
```

```yaml
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required:
              - query
            properties:
              query:
                type: string
                maxLength: 1000
              asset_context:
                type: object
                description: "Current asset state for context"
                properties:
                  asset_id:
                    type: string
                  sensor_values:
                    type: object
                  active_alerts:
                    type: array
              document_types:
                type: array
                items:
                  type: string
                description: "Filter by document types"
              top_k:
                type: integer
                default: 5
                maximum: 20
              include_sources:
                type: boolean
                default: true
    responses:
      '200':
        description: RAG response
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/RAGResponse'

/rag/feedback:
  post:
    summary: Submit feedback on RAG response
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required:
              - query_id
              - feedback
```

```yaml
              properties:
                query_id:
                  type: string
                  format: uuid
                feedback:
                  type: string
                  enum: [helpful, not_helpful]
                notes:
                  type: string
      responses:
        '200':
          description: Feedback recorded

components:
  schemas:
    RAGResponse:
      type: object
      properties:
        query_id:
          type: string
          format: uuid
        query:
          type: string
        response:
          type: string
        confidence:
          type: number
        sources:
          type: array
          items:
            type: object
            properties:
              document_id:
                type: string
              document_title:
                type: string
              chunk_preview:
                type: string
              relevance_score:
                type: number
              page_number:
                type: integer
        asset_context_used:
          type: boolean
        latency_ms:
          type: integer
```

4. DATA MODELS AND CLASSES

4.1 Python Data Models (Pydantic)

```python
# models/asset.py
from pydantic import BaseModel, Field
from typing import Optional, List, Dict, Any
from datetime import datetime
from uuid import UUID
from enum import Enum

class AssetType(str, Enum):
    PUMP = "pump"
    COMPRESSOR = "compressor"
    MOTOR = "motor"
    VALVE = "valve"
    TANK = "tank"

class AssetStatus(str, Enum):
    OPERATIONAL = "operational"
    DEGRADED = "degraded"
    FAILED = "failed"
    MAINTENANCE = "maintenance"

class Criticality(str, Enum):
    LOW = "low"
    MEDIUM = "medium"
    HIGH = "high"
    CRITICAL = "critical"

class AssetBase(BaseModel):
    asset_tag: str = Field(..., max_length=50)
    name: str = Field(..., max_length=255)
    asset_type: AssetType
    manufacturer: Optional[str] = None
    model: Optional[str] = None
    serial_number: Optional[str] = None
    location_id: UUID
    parent_asset_id: Optional[UUID] = None
    criticality: Criticality = Criticality.MEDIUM
    specifications: Optional[Dict[str, Any]] = None

class AssetCreate(AssetBase):
    pass

class AssetUpdate(BaseModel):
    name: Optional[str] = None
    status: Optional[AssetStatus] = None
    criticality: Optional[Criticality] = None
    specifications: Optional[Dict[str, Any]] = None

class Asset(AssetBase):
    asset_id: UUID
    status: AssetStatus = AssetStatus.OPERATIONAL
```

```
    created_at: datetime
    updated_at: datetime

    class Config:
        from_attributes = True

class AssetHealth(BaseModel):
    asset_id: UUID
    health_index: float = Field(..., ge=0, le=1)
    status: AssetStatus
    rul_days: Optional[int] = Field(None, description="Remaining Useful Life")
    anomaly_score: float = Field(0.0, ge=0, le=1)
    last_prediction: Optional[datetime] = None
    active_alerts: int = 0
    sensors: List[Dict[str, Any]] = []


# models/sensor.py
class SensorType(str, Enum):
    VIBRATION = "vibration"
    TEMPERATURE = "temperature"
    PRESSURE = "pressure"
    FLOW = "flow"
    LEVEL = "level"
    CURRENT = "current"

class SensorStatus(str, Enum):
    ACTIVE = "active"
    INACTIVE = "inactive"
    FAULTY = "faulty"
    CALIBRATING = "calibrating"

class SensorBase(BaseModel):
    sensor_tag: str = Field(..., max_length=50)
    asset_id: UUID
    sensor_type: SensorType
    manufacturer: Optional[str] = None
    model: Optional[str] = None
    unit_of_measure: str
    min_value: Optional[float] = None
    max_value: Optional[float] = None
    alarm_low: Optional[float] = None
    alarm_high: Optional[float] = None
    sampling_rate_hz: Optional[float] = None
    communication_protocol: Optional[str] = None
    opc_node_id: Optional[str] = None

class Sensor(SensorBase):
    sensor_id: UUID
    status: SensorStatus = SensorStatus.ACTIVE
    calibration_date: Optional[datetime] = None
    created_at: datetime

    class Config:
```

```
        from_attributes = True

class SensorReading(BaseModel):
    sensor_id: UUID
    timestamp: datetime
    value: float
    quality: int = Field(..., ge=0, le=2)  # 0=bad, 1=uncertain, 2=good
    raw_value: Optional[float] = None


# models/alert.py
class AlertSeverity(str, Enum):
    INFO = "info"
    WARNING = "warning"
    CRITICAL = "critical"
    EMERGENCY = "emergency"

class AlertStatus(str, Enum):
    OPEN = "open"
    ACKNOWLEDGED = "acknowledged"
    RESOLVED = "resolved"
    FALSE_POSITIVE = "false_positive"

class AlertCreate(BaseModel):
    definition_id: UUID
    asset_id: UUID
    sensor_id: Optional[UUID] = None
    severity: AlertSeverity
    title: str
    description: Optional[str] = None
    confidence_score: float = Field(..., ge=0, le=1)
    model_version: Optional[str] = None
    related_data: Optional[Dict[str, Any]] = None

class Alert(AlertCreate):
    alert_id: UUID
    triggered_at: datetime
    acknowledged_at: Optional[datetime] = None
    acknowledged_by: Optional[UUID] = None
    resolved_at: Optional[datetime] = None
    resolved_by: Optional[UUID] = None
    resolution_notes: Optional[str] = None
    status: AlertStatus = AlertStatus.OPEN
    created_at: datetime

    class Config:
        from_attributes = True


# models/prediction.py
class PredictionOutput(BaseModel):
    asset_id: UUID
    model_name: str
    model_version: str
```

```
    prediction_type: str  # rul, anomaly, classification
    rul_days: Optional[int] = None
    anomaly_score: Optional[float] = None
    health_index: Optional[float] = None
    confidence: float
    predicted_failure_mode: Optional[str] = None
    features_used: Dict[str, float]
    predicted_at: datetime


# models/rag.py
class RAGQuery(BaseModel):
    query: str = Field(..., max_length=1000)
    asset_context: Optional[Dict[str, Any]] = None
    document_types: Optional[List[str]] = None
    top_k: int = Field(5, ge=1, le=20)
    include_sources: bool = True

class RAGSource(BaseModel):
    document_id: UUID
    document_title: str
    chunk_preview: str
    relevance_score: float
    page_number: Optional[int] = None

class RAGResponse(BaseModel):
    query_id: UUID
    query: str
    response: str
    confidence: float
    sources: List[RAGSource]
    asset_context_used: bool
    latency_ms: int
```

4.2 ML Feature Classes

```
# ml/features.py
from dataclasses import dataclass
from typing import List, Optional
import numpy as np

@dataclass
class VibrationFeatures:
    """Features extracted from vibration waveform"""
    # Time domain
    rms: float
    peak: float
    peak_to_peak: float
    crest_factor: float
    kurtosis: float
    skewness: float

    # Frequency domain
    fft_1x: float  # 1x rotational frequency amplitude
```

```
    fft_2x: float  # 2x
    fft_3x: float  # 3x

    # Bearing frequencies
    bpfo: float  # Ball Pass Frequency Outer
    bpfi: float  # Ball Pass Frequency Inner
    bsf: float   # Ball Spin Frequency
    ftf: float   # Fundamental Train Frequency

    # Envelope analysis
    envelope_rms: float
    envelope_kurtosis: float

    @classmethod
    def from_waveform(cls, waveform: np.ndarray,
                      sampling_rate: float,
                      rpm: float,
                      bearing_params: dict) -> 'VibrationFeatures':
        """Extract features from raw waveform"""
        # Time domain
        rms = np.sqrt(np.mean(waveform**2))
        peak = np.max(np.abs(waveform))
        peak_to_peak = np.max(waveform) - np.min(waveform)
        crest_factor = peak / rms if rms > 0 else 0
        kurtosis = scipy.stats.kurtosis(waveform)
        skewness = scipy.stats.skew(waveform)

        # FFT
        fft_result = np.fft.rfft(waveform)
        fft_freqs = np.fft.rfftfreq(len(waveform), 1/sampling_rate)
        fft_magnitude = np.abs(fft_result)

        # Rotational frequencies
        rot_freq = rpm / 60
        fft_1x = cls._get_amplitude_at_freq(fft_freqs, fft_magnitude, rot_freq)
        fft_2x = cls._get_amplitude_at_freq(fft_freqs, fft_magnitude, 2*rot_freq)
        fft_3x = cls._get_amplitude_at_freq(fft_freqs, fft_magnitude, 3*rot_freq)

        # Bearing frequencies
        bpfo = cls._get_amplitude_at_freq(fft_freqs, fft_magnitude,
                                          bearing_params['bpfo'] * rot_freq)
        bpfi = cls._get_amplitude_at_freq(fft_freqs, fft_magnitude,
                                          bearing_params['bpfi'] * rot_freq)
        bsf = cls._get_amplitude_at_freq(fft_freqs, fft_magnitude,
                                         bearing_params['bsf'] * rot_freq)
        ftf = cls._get_amplitude_at_freq(fft_freqs, fft_magnitude,
                                         bearing_params['ftf'] * rot_freq)

        # Envelope analysis
        envelope = cls._compute_envelope(waveform)
        envelope_rms = np.sqrt(np.mean(envelope**2))
        envelope_kurtosis = scipy.stats.kurtosis(envelope)

        return cls(
```

```
            rms=rms, peak=peak, peak_to_peak=peak_to_peak,
            crest_factor=crest_factor, kurtosis=kurtosis, skewness=skewness,
            fft_1x=fft_1x, fft_2x=fft_2x, fft_3x=fft_3x,
            bpfo=bpfo, bpfi=bpfi, bsf=bsf, ftf=ftf,
            envelope_rms=envelope_rms, envelope_kurtosis=envelope_kurtosis
        )

    @staticmethod
    def _get_amplitude_at_freq(freqs, magnitudes, target_freq, bandwidth=5):
        """Get amplitude at target frequency with bandwidth tolerance"""
        mask = (freqs >= target_freq - bandwidth) & (freqs <= target_freq + bandwidth)
        if np.any(mask):
            return np.max(magnitudes[mask])
        return 0.0

    @staticmethod
    def _compute_envelope(signal):
        """Compute envelope using Hilbert transform"""
        analytic_signal = scipy.signal.hilbert(signal)
        return np.abs(analytic_signal)

    def to_array(self) -> np.ndarray:
        """Convert to numpy array for ML model input"""
        return np.array([
            self.rms, self.peak, self.peak_to_peak, self.crest_factor,
            self.kurtosis, self.skewness, self.fft_1x, self.fft_2x, self.fft_3x,
            self.bpfo, self.bpfi, self.bsf, self.ftf,
            self.envelope_rms, self.envelope_kurtosis
        ])


@dataclass
class MultiSensorFeatures:
    """Combined features from multiple sensor types"""
    asset_id: str
    timestamp: datetime

    # Vibration (from 2 sensors)
    vib_x_rms: float
    vib_x_kurtosis: float
    vib_y_rms: float
    vib_y_kurtosis: float

    # Temperature
    bearing_temp: float
    motor_temp: float
    ambient_temp: float
    temp_rise: float  # bearing - ambient

    # Electrical
    current_rms: float
    current_thd: float  # Total Harmonic Distortion
    power_factor: float
```

```
    # Derived
    load_factor: float
    efficiency_estimate: float

    def to_dict(self) -> dict:
        return {k: v for k, v in self.__dict__.items()}
```

---

5. ALGORITHM SPECIFICATIONS

5.1 Anomaly Detection Algorithm

```
# ml/anomaly_detector.py
"""
Autoencoder-based Anomaly Detection for Sensor Data

Architecture:
- Input: 15 features (vibration + temperature + electrical)
- Encoder: 15 -> 10 -> 5 -> 3 (latent space)
- Decoder: 3 -> 5 -> 10 -> 15
- Loss: MSE reconstruction loss
- Anomaly Score: reconstruction error normalized by training distribution
"""

import torch
import torch.nn as nn
import numpy as np
from typing import Tuple

class AnomalyAutoencoder(nn.Module):
    def __init__(self, input_dim: int = 15, latent_dim: int = 3):
        super().__init__()

        # Encoder
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, 10),
            nn.ReLU(),
            nn.BatchNorm1d(10),
            nn.Dropout(0.2),
            nn.Linear(10, 5),
            nn.ReLU(),
            nn.BatchNorm1d(5),
            nn.Linear(5, latent_dim)
        )

        # Decoder
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 5),
            nn.ReLU(),
            nn.BatchNorm1d(5),
            nn.Linear(5, 10),
            nn.ReLU(),
            nn.BatchNorm1d(10),
```

```
            nn.Dropout(0.2),
            nn.Linear(10, input_dim)
        )

    def forward(self, x: torch.Tensor) -> Tuple[torch.Tensor, torch.Tensor]:
        latent = self.encoder(x)
        reconstructed = self.decoder(latent)
        return reconstructed, latent

    def compute_anomaly_score(self, x: torch.Tensor,
                              threshold_mean: float,
                              threshold_std: float) -> Tuple[float, bool]:
        """
        Compute anomaly score based on reconstruction error

        Returns:
            score: Normalized anomaly score (0-1)
            is_anomaly: True if score > threshold
        """
        self.eval()
        with torch.no_grad():
            reconstructed, _ = self.forward(x)
            mse = torch.mean((x - reconstructed) ** 2, dim=1)

            # Normalize using training distribution
            z_score = (mse - threshold_mean) / threshold_std

            # Convert to 0-1 scale using sigmoid
            score = torch.sigmoid(z_score)

            # Threshold at 0.95 (2 standard deviations)
            is_anomaly = score > 0.95

            return score.item(), is_anomaly.item()


class AnomalyDetector:
    """
    Production-ready anomaly detector with calibration and drift detection
    """
    def __init__(self, model_path: str, config: dict):
        self.model = AnomalyAutoencoder(
            input_dim=config['input_dim'],
            latent_dim=config['latent_dim']
        )
        self.model.load_state_dict(torch.load(model_path))
        self.model.eval()

        # Calibration parameters (from training data)
        self.threshold_mean = config['threshold_mean']
        self.threshold_std = config['threshold_std']

        # Feature normalization
        self.feature_mean = np.array(config['feature_mean'])
```

```
        self.feature_std = np.array(config['feature_std'])

        # Moving average for drift detection
        self.score_history = []
        self.history_window = 1000

    def preprocess(self, features: np.ndarray) -> torch.Tensor:
        """Normalize features using training statistics"""
        normalized = (features - self.feature_mean) / self.feature_std
        return torch.FloatTensor(normalized).unsqueeze(0)

    def predict(self, features: np.ndarray) -> dict:
        """
        Run anomaly detection on feature vector

        Args:
            features: numpy array of shape (15,)

        Returns:
            dict with anomaly_score, is_anomaly, confidence
        """
        x = self.preprocess(features)
        score, is_anomaly = self.model.compute_anomaly_score(
            x, self.threshold_mean, self.threshold_std
        )

        # Update history for drift detection
        self.score_history.append(score)
        if len(self.score_history) > self.history_window:
            self.score_history.pop(0)

        # Calculate confidence based on score stability
        if len(self.score_history) >= 10:
            recent_std = np.std(self.score_history[-10:])
            confidence = 1.0 - min(recent_std * 10, 1.0)
        else:
            confidence = 0.5

        return {
            'anomaly_score': score,
            'is_anomaly': is_anomaly,
            'confidence': confidence,
            'threshold': 0.95
        }

    def detect_drift(self) -> bool:
        """Detect if model is drifting based on score distribution"""
        if len(self.score_history) < 100:
            return False

        recent = self.score_history[-100:]
        baseline = self.score_history[:100]

        # KS test for distribution shift
```

```
        from scipy.stats import ks_2samp
        statistic, p_value = ks_2samp(recent, baseline)

        return p_value < 0.05  # Significant drift detected
```

5.2 RUL Prediction Algorithm

```
# ml/rul_predictor.py
"""
Remaining Useful Life (RUL) Prediction using CNN-LSTM hybrid

Architecture:
- Input: Sequence of 50 timesteps, 15 features each
- CNN layers: Extract local patterns
- LSTM layers: Capture temporal dependencies
- Output: RUL in days (regression)
"""

import torch
import torch.nn as nn
from typing import Tuple

class CNNLSTM_RUL(nn.Module):
    def __init__(self,
                 n_features: int = 15,
                 seq_length: int = 50,
                 cnn_filters: int = 64,
                 lstm_units: int = 64,
                 dropout: float = 0.3):
        super().__init__()

        # CNN layers for feature extraction
        self.cnn = nn.Sequential(
            nn.Conv1d(n_features, cnn_filters, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.BatchNorm1d(cnn_filters),
            nn.MaxPool1d(2),
            nn.Conv1d(cnn_filters, cnn_filters*2, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.BatchNorm1d(cnn_filters*2),
            nn.MaxPool1d(2),
            nn.Dropout(dropout)
        )

        # Calculate CNN output size
        cnn_out_length = seq_length // 4  # After two MaxPool1d(2)

        # LSTM layers for temporal modeling
        self.lstm = nn.LSTM(
            input_size=cnn_filters*2,
            hidden_size=lstm_units,
            num_layers=2,
            batch_first=True,
            dropout=dropout,
```

```
            bidirectional=True
        )

        # Attention mechanism
        self.attention = nn.Sequential(
            nn.Linear(lstm_units*2, lstm_units),
            nn.Tanh(),
            nn.Linear(lstm_units, 1)
        )

        # Output layers
        self.fc = nn.Sequential(
            nn.Linear(lstm_units*2, 32),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(32, 1)  # RUL output
        )

    def forward(self, x: torch.Tensor) -> Tuple[torch.Tensor, torch.Tensor]:
        """
        Forward pass

        Args:
            x: Input tensor of shape (batch, seq_length, n_features)

        Returns:
            rul: Predicted RUL (batch, 1)
            attention_weights: Attention weights (batch, seq_length)
        """
        # Transpose for CNN: (batch, features, seq_length)
        x = x.transpose(1, 2)

        # CNN feature extraction
        cnn_out = self.cnn(x)

        # Transpose back for LSTM: (batch, seq_length, features)
        cnn_out = cnn_out.transpose(1, 2)

        # LSTM
        lstm_out, _ = self.lstm(cnn_out)

        # Attention
        attention_scores = self.attention(lstm_out)
        attention_weights = torch.softmax(attention_scores, dim=1)

        # Weighted sum
        context = torch.sum(attention_weights * lstm_out, dim=1)

        # Output
        rul = self.fc(context)

        return rul, attention_weights.squeeze(-1)
```

```python
class RULPredictor:
    """Production RUL predictor with uncertainty estimation"""

    def __init__(self, model_path: str, config: dict):
        self.model = CNNLSTM_RUL(
            n_features=config['n_features'],
            seq_length=config['seq_length'],
            cnn_filters=config['cnn_filters'],
            lstm_units=config['lstm_units']
        )
        self.model.load_state_dict(torch.load(model_path))
        self.model.eval()

        self.seq_length = config['seq_length']
        self.feature_mean = config['feature_mean']
        self.feature_std = config['feature_std']
        self.rul_max = config['rul_max']  # Normalization factor

        # For uncertainty estimation (MC Dropout)
        self.n_mc_samples = 20

    def preprocess_sequence(self, features_seq: np.ndarray) -> torch.Tensor:
        """Normalize and prepare sequence for model"""
        normalized = (features_seq - self.feature_mean) / self.feature_std
        return torch.FloatTensor(normalized).unsqueeze(0)

    def predict(self, features_seq: np.ndarray) -> dict:
        """
        Predict RUL with uncertainty estimation

        Args:
            features_seq: numpy array of shape (seq_length, n_features)

        Returns:
            dict with rul_days, confidence_interval, attention_weights
        """
        x = self.preprocess_sequence(features_seq)

        # Single prediction
        with torch.no_grad():
            rul_pred, attention = self.model(x)
            rul_days = rul_pred.item() * self.rul_max

        # MC Dropout for uncertainty
        self.model.train()  # Enable dropout
        mc_predictions = []
        for _ in range(self.n_mc_samples):
            with torch.no_grad():
                pred, _ = self.model(x)
                mc_predictions.append(pred.item() * self.rul_max)
        self.model.eval()

        mc_predictions = np.array(mc_predictions)
        rul_mean = np.mean(mc_predictions)
```

```
        rul_std = np.std(mc_predictions)

        # 95% confidence interval
        ci_lower = max(0, rul_mean - 1.96 * rul_std)
        ci_upper = rul_mean + 1.96 * rul_std

        # Confidence based on uncertainty
        confidence = 1.0 - min(rul_std / rul_mean, 1.0) if rul_mean > 0 else 0.0

        return {
            'rul_days': int(rul_mean),
            'rul_std': rul_std,
            'ci_lower': int(ci_lower),
            'ci_upper': int(ci_upper),
            'confidence': confidence,
            'attention_weights': attention.numpy().tolist()
        }
```

5.3 RAG Pipeline Algorithm

```
# rag/pipeline.py
"""
RAG Pipeline for Oil & Gas Knowledge Assistant

Pipeline:
1. Query preprocessing (expand abbreviations, add context)
2. Embedding generation (BGE-large)
3. Vector search (Milvus)
4. Reranking (Cross-encoder)
5. Context assembly (with IoT context)
6. LLM generation (Llama 3.1)
7. Response post-processing (citations, formatting)
"""

from typing import List, Dict, Optional, Tuple
import numpy as np
from sentence_transformers import SentenceTransformer, CrossEncoder
from pymilvus import connections, Collection
from langchain.prompts import ChatPromptTemplate
from langchain.llms import VLLM
import hashlib
import redis

class RAGPipeline:
    def __init__(self, config: dict):
        # Models
        self.embedder = SentenceTransformer(config['embedding_model'])
        self.reranker = CrossEncoder(config['reranker_model'])

        # Milvus connection
        connections.connect(
            alias="default",
            host=config['milvus_host'],
            port=config['milvus_port']
```

```
        )
        self.collection = Collection(config['collection_name'])
        self.collection.load()

        # LLM
        self.llm = VLLM(
            model=config['llm_model'],
            tensor_parallel_size=config['tensor_parallel_size'],
            max_new_tokens=config['max_new_tokens'],
            temperature=config['temperature']
        )

        # Cache
        self.cache = redis.Redis(
            host=config['redis_host'],
            port=config['redis_port'],
            db=config['redis_db']
        )
        self.cache_ttl = config['cache_ttl_seconds']

        # Prompt template
        self.system_prompt = """You are an expert Oil & Gas engineer assistant.
Answer questions based ONLY on the provided context. If the context doesn't
contain enough information, say so clearly. Always cite your sources using
[Source: document_title, page X] format.

Current Equipment Context (if provided):
{asset_context}

Retrieved Documents:
{context}

User Question: {question}

Answer:"""

        self.prompt = ChatPromptTemplate.from_template(self.system_prompt)

        # O&G abbreviation expansion
        self.abbreviations = {
            'P&ID': 'Piping and Instrumentation Diagram',
            'SOP': 'Standard Operating Procedure',
            'ESD': 'Emergency Shutdown',
            'PSV': 'Pressure Safety Valve',
            'MOC': 'Management of Change',
            'RBI': 'Risk Based Inspection',
            'API': 'American Petroleum Institute',
            'ASME': 'American Society of Mechanical Engineers'
        }

    def preprocess_query(self, query: str) -> str:
        """Expand abbreviations and clean query"""
        processed = query
        for abbr, expansion in self.abbreviations.items():
```

```
            processed = processed.replace(abbr, f"{abbr} ({expansion})")
        return processed

    def get_cache_key(self, query: str, asset_id: Optional[str]) -> str:
        """Generate cache key from query"""
        key_string = f"{query}:{asset_id or 'none'}"
        return hashlib.sha256(key_string.encode()).hexdigest()

    def embed_query(self, query: str) -> np.ndarray:
        """Generate query embedding"""
        return self.embedder.encode(query, normalize_embeddings=True)

    def vector_search(self, embedding: np.ndarray,
                      top_k: int = 20,
                      doc_types: Optional[List[str]] = None) -> List[dict]:
        """Search Milvus for relevant chunks"""
        search_params = {"metric_type": "COSINE", "params": {"nprobe": 10}}

        # Build filter expression
        expr = None
        if doc_types:
            doc_type_list = ', '.join([f'"{t}"' for t in doc_types])
            expr = f"document_type in [{doc_type_list}]"

        results = self.collection.search(
            data=[embedding.tolist()],
            anns_field="embedding",
            param=search_params,
            limit=top_k,
            expr=expr,
            output_fields=["chunk_id", "document_id", "document_type", "content_preview"]
        )

        chunks = []
        for hit in results[0]:
            chunks.append({
                'chunk_id': hit.entity.get('chunk_id'),
                'document_id': hit.entity.get('document_id'),
                'document_type': hit.entity.get('document_type'),
                'content': hit.entity.get('content_preview'),
                'score': hit.score
            })

        return chunks

    def rerank(self, query: str, chunks: List[dict], top_k: int = 5) -> List[dict]:
        """Rerank chunks using cross-encoder"""
        if not chunks:
            return []

        pairs = [(query, chunk['content']) for chunk in chunks]
        scores = self.reranker.predict(pairs)

        for i, score in enumerate(scores):
```

```
            chunks[i]['rerank_score'] = float(score)

        reranked = sorted(chunks, key=lambda x: x['rerank_score'], reverse=True)
        return reranked[:top_k]

    def format_asset_context(self, asset_context: Optional[dict]) -> str:
        """Format current asset state for prompt"""
        if not asset_context:
            return "No current equipment context provided."

        lines = []
        if 'asset_id' in asset_context:
            lines.append(f"Asset ID: {asset_context['asset_id']}")

        if 'sensor_values' in asset_context:
            lines.append("Current Sensor Readings:")
            for sensor, value in asset_context['sensor_values'].items():
                lines.append(f"  - {sensor}: {value}")

        if 'active_alerts' in asset_context:
            if asset_context['active_alerts']:
                lines.append("Active Alerts:")
                for alert in asset_context['active_alerts']:
                    lines.append(f"  - {alert}")
            else:
                lines.append("No active alerts.")

        return '\n'.join(lines)

    def format_context(self, chunks: List[dict]) -> str:
        """Format retrieved chunks for prompt"""
        context_parts = []
        for i, chunk in enumerate(chunks, 1):
            context_parts.append(
                f"[Document {i}: {chunk.get('document_title', 'Unknown')}]\n"
                f"{chunk['content']}\n"
            )
        return '\n---\n'.join(context_parts)

    def generate_response(self, query: str, context: str,
                          asset_context_str: str) -> Tuple[str, dict]:
        """Generate LLM response"""
        prompt_text = self.prompt.format(
            question=query,
            context=context,
            asset_context=asset_context_str
        )

        response = self.llm.generate([prompt_text])
        generated_text = response.generations[0][0].text

        # Extract token counts
        usage = {
            'prompt_tokens': len(prompt_text.split()),  # Approximate
```

```
            'completion_tokens': len(generated_text.split())
        }

        return generated_text, usage

    def query(self,
              query: str,
              asset_context: Optional[dict] = None,
              doc_types: Optional[List[str]] = None,
              top_k: int = 5,
              use_cache: bool = True) -> dict:
        """
        Full RAG pipeline execution

        Args:
            query: User question
            asset_context: Current asset state for context
            doc_types: Filter by document types
            top_k: Number of chunks to use
            use_cache: Whether to use cache

        Returns:
            dict with response, sources, latency, etc.
        """
        import time
        start_time = time.time()

        # Check cache
        cache_key = self.get_cache_key(query, asset_context.get('asset_id') if asset_context else None)
        if use_cache:
            cached = self.cache.get(cache_key)
            if cached:
                result = json.loads(cached)
                result['cached'] = True
                return result

        # Preprocess
        processed_query = self.preprocess_query(query)

        # Embed
        embedding = self.embed_query(processed_query)

        # Search
        chunks = self.vector_search(embedding, top_k=top_k*4, doc_types=doc_types)

        # Enrich with document metadata
        chunks = self._enrich_chunks(chunks)

        # Rerank
        reranked = self.rerank(processed_query, chunks, top_k=top_k)

        # Format context
        context = self.format_context(reranked)
        asset_context_str = self.format_asset_context(asset_context)
```

```
        # Generate
        response, usage = self.generate_response(query, context, asset_context_str)

        # Prepare result
        latency_ms = int((time.time() - start_time) * 1000)

        sources = [{
            'document_id': c['document_id'],
            'document_title': c.get('document_title', 'Unknown'),
            'chunk_preview': c['content'][:200],
            'relevance_score': c['rerank_score'],
            'page_number': c.get('page_number')
        } for c in reranked]

        result = {
            'response': response,
            'sources': sources,
            'latency_ms': latency_ms,
            'tokens': usage,
            'asset_context_used': asset_context is not None,
            'cached': False
        }

        # Cache result
        if use_cache:
            self.cache.setex(
                cache_key,
                self.cache_ttl,
                json.dumps(result)
            )

        return result

    def _enrich_chunks(self, chunks: List[dict]) -> List[dict]:
        """Fetch additional metadata from PostgreSQL"""
        # Implementation would query PostgreSQL for document titles, page numbers, etc.
        # Simplified here
        return chunks
```

---

6. CONFIGURATION SPECIFICATIONS

6.1 Application Configuration

```yaml
# config/app.yaml
app:
  name: "OilGas AI Monitoring"
  version: "1.0.0"
  environment: "production"  # development, staging, production
  debug: false
  log_level: "INFO"
```

```yaml
server:
  host: "0.0.0.0"
  port: 8000
  workers: 4
  timeout: 60
  max_request_size: "10MB"

database:
  postgresql:
    host: "${POSTGRES_HOST}"
    port: 5432
    database: "oilgas_ai"
    user: "${POSTGRES_USER}"
    password: "${POSTGRES_PASSWORD}"
    pool_size: 20
    max_overflow: 10
    ssl_mode: "require"

  influxdb:
    url: "${INFLUXDB_URL}"
    token: "${INFLUXDB_TOKEN}"
    org: "oilgas"
    bucket: "sensor_data"
    timeout: 10000

  redis:
    host: "${REDIS_HOST}"
    port: 6379
    db: 0
    password: "${REDIS_PASSWORD}"
    ssl: true

milvus:
  host: "${MILVUS_HOST}"
  port: 19530
  collection: "document_embeddings"

kafka:
  bootstrap_servers: "${KAFKA_BROKERS}"
  security_protocol: "SASL_SSL"
  sasl_mechanism: "SCRAM-SHA-512"
  sasl_username: "${KAFKA_USER}"
  sasl_password: "${KAFKA_PASSWORD}"
  consumer_group: "ai_platform"

ml:
  model_registry: "s3://models/registry"
  inference_timeout: 5000  # ms
  batch_size: 32

  anomaly_detector:
    model_path: "s3://models/anomaly/v1.2.0/model.onnx"
    threshold_mean: 0.023
    threshold_std: 0.015
```

```yaml
  rul_predictor:
    model_path: "s3://models/rul/v1.1.0/model.pt"
    seq_length: 50
    rul_max: 365

rag:
  embedding_model: "BAAI/bge-large-en-v1.5"
  reranker_model: "BAAI/bge-reranker-large"
  llm_model: "meta-llama/Llama-3.1-70B"
  max_tokens: 2048
  temperature: 0.1
  top_k: 5
  cache_ttl: 3600  # seconds

alerts:
  notification:
    email:
      smtp_host: "${SMTP_HOST}"
      smtp_port: 587
      from_address: "alerts@oilgas-ai.com"
    sms:
      provider: "twilio"
      from_number: "${TWILIO_NUMBER}"
    push:
      firebase_project: "${FIREBASE_PROJECT}"

  escalation:
    warning_minutes: 30
    critical_minutes: 15
    emergency_minutes: 5

security:
  jwt:
    secret: "${JWT_SECRET}"
    algorithm: "HS256"
    expiry_hours: 24

  oauth:
    provider: "keycloak"
    issuer_url: "${KEYCLOAK_URL}/realms/oilgas"
    client_id: "${OAUTH_CLIENT_ID}"
    client_secret: "${OAUTH_CLIENT_SECRET}"

  cors:
    allowed_origins:
      - "https://dashboard.oilgas-ai.internal"
      - "https://mobile.oilgas-ai.internal"
    allowed_methods: ["GET", "POST", "PUT", "DELETE"]
    allowed_headers: ["*"]

observability:
  prometheus:
    enabled: true
```

```yaml
    port: 9090

  jaeger:
    enabled: true
    endpoint: "${JAEGER_ENDPOINT}"
    service_name: "ai-platform"

  logging:
    format: "json"
    output: "stdout"
    level: "INFO"
```

6.2 Edge Node Configuration

```yaml
# config/edge.yaml
edge:
  node_id: "${EDGE_NODE_ID}"
  location_id: "${LOCATION_ID}"

  hardware:
    type: "jetson_agx_orin"
    gpu_memory: "64GB"

  network:
    mqtt:
      broker: "${MQTT_BROKER}"
      port: 8883
      use_tls: true
      client_id: "${EDGE_NODE_ID}"
      username: "${MQTT_USER}"
      password: "${MQTT_PASSWORD}"
      topics:
        publish:
          - "edge/${EDGE_NODE_ID}/predictions"
          - "edge/${EDGE_NODE_ID}/alerts"
          - "edge/${EDGE_NODE_ID}/health"
        subscribe:
          - "edge/${EDGE_NODE_ID}/config"
          - "edge/${EDGE_NODE_ID}/models"

    opc_ua:
      endpoint: "opc.tcp://localhost:4840"
      security_mode: "SignAndEncrypt"
      security_policy: "Basic256Sha256"
      user: "${OPC_USER}"
      password: "${OPC_PASSWORD}"

  inference:
    models:
      anomaly_detector:
        path: "/models/anomaly_detector.engine"
        batch_size: 1
        precision: "fp16"
```

```yaml
    ppe_detector:
      path: "/models/ppe_yolov8.engine"
      batch_size: 1
      precision: "fp16"
      input_size: [640, 640]
      conf_threshold: 0.5
      nms_threshold: 0.45

    corrosion_detector:
      path: "/models/corrosion_yolov8.engine"
      batch_size: 1
      precision: "fp16"
      input_size: [1280, 1280]

  video:
    cameras:
      - id: "cam_001"
        uri: "rtsp://192.168.1.100:554/stream1"
        fps: 15
        resolution: [1920, 1080]
        models: ["ppe_detector"]

      - id: "cam_002"
        uri: "rtsp://192.168.1.101:554/stream1"
        fps: 5
        resolution: [1920, 1080]
        models: ["corrosion_detector"]

  vibration:
    sampling_rate: 25600  # Hz
    samples_per_batch: 8192
    window_overlap: 0.5
    fft_bins: 4096

storage:
  local_buffer: "/data/buffer"
  buffer_size_gb: 100
  retention_hours: 48

health:
  heartbeat_interval: 60  # seconds
  metrics_interval: 10   # seconds
```

---

7. MESSAGE FORMATS

7.1 Kafka Message Schemas

```json
// sensor_readings topic
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "required": ["sensor_id", "timestamp", "value"],
```

```
  "properties": {
    "sensor_id": {
      "type": "string",
      "format": "uuid"
    },
    "asset_id": {
      "type": "string",
      "format": "uuid"
    },
    "timestamp": {
      "type": "string",
      "format": "date-time"
    },
    "value": {
      "type": "number"
    },
    "quality": {
      "type": "integer",
      "enum": [0, 1, 2]
    },
    "unit": {
      "type": "string"
    }
  }
}

// predictions topic
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "required": ["prediction_id", "asset_id", "model_name", "timestamp"],
  "properties": {
    "prediction_id": {
      "type": "string",
      "format": "uuid"
    },
    "asset_id": {
      "type": "string",
      "format": "uuid"
    },
    "model_name": {
      "type": "string"
    },
    "model_version": {
      "type": "string"
    },
    "timestamp": {
      "type": "string",
      "format": "date-time"
    },
    "prediction_type": {
      "type": "string",
      "enum": ["rul", "anomaly", "classification"]
    },
```

```
    "output": {
      "type": "object",
      "properties": {
        "rul_days": {"type": "integer"},
        "anomaly_score": {"type": "number"},
        "health_index": {"type": "number"},
        "confidence": {"type": "number"}
      }
    },
    "input_features": {
      "type": "object"
    }
  }
}

// alerts topic
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "required": ["alert_id", "asset_id", "severity", "triggered_at"],
  "properties": {
    "alert_id": {
      "type": "string",
      "format": "uuid"
    },
    "definition_id": {
      "type": "string",
      "format": "uuid"
    },
    "asset_id": {
      "type": "string",
      "format": "uuid"
    },
    "sensor_id": {
      "type": "string",
      "format": "uuid"
    },
    "severity": {
      "type": "string",
      "enum": ["info", "warning", "critical", "emergency"]
    },
    "title": {
      "type": "string"
    },
    "description": {
      "type": "string"
    },
    "confidence_score": {
      "type": "number",
      "minimum": 0,
      "maximum": 1
    },
    "triggered_at": {
      "type": "string",
```

```
      "format": "date-time"
    },
    "related_data": {
      "type": "object"
    }
  }
}
```

---

8. APPENDIX

8.1 Error Codes

Code	HTTP Status	Description
ERR_AUTH_001	401	Invalid or expired token
ERR_AUTH_002	403	Insufficient permissions
ERR_ASSET_001	404	Asset not found
ERR_ASSET_002	409	Asset tag already exists
ERR_SENSOR_001	404	Sensor not found
ERR_MODEL_001	500	Model inference failed
ERR_MODEL_002	503	Model not loaded
ERR_RAG_001	500	RAG pipeline error
ERR_RAG_002	400	Query too long
ERR_DB_001	500	Database connection error
ERR_KAFKA_001	500	Kafka publish failed

8.2 Monitoring Metrics

Metric	Type	Labels	Description
`api_request_duration_seconds`	Histogram	method, endpoint, status	API latency
`model_inference_duration_seconds`	Histogram	model_name, version	Inference latency
`model_prediction_total`	Counter	model_name, asset_type	Total predictions
`alert_triggered_total`	Counter	severity, alert_type	Alerts generated
`sensor_reading_lag_seconds`	Gauge	sensor_type	Data freshness
`edge_node_health`	Gauge	node_id	Edge node status
`rag_query_duration_seconds`	Histogram	cached	RAG query latency

Document Approval

Role	Name	Signature	Date
Technical Lead			
Solution Architect			
Database Admin			

END OF LLD DOCUMENT